

클라우드 컴퓨팅 인프라를 위한 16-비트 프로세서 및 컴퓨터 구조의 논리 회로 합성과 형식적 명세

(Logic Synthesis and Formal Specification of a 16-Bit Computer Architecture for Cloud Computing Infrastructure)

인수 이 (Insu YI)

컴퓨터 공학과 (Department of Computer Science and Engineering)

실험일: 2026년 7월 22일 — 작성일: 2026년 7월 28일

이메일: ii3289857@gmail.com

Abstract—본 논문은 추후 구축 예정인 클라우드 컴퓨팅 시스템의 모듈형 하드웨어 베이스라인으로 활용하기 위한 16비트 연산 및 제어 프로세서(Computer Architecture)의 논리 회로 설계 및 형식적 명세를 제안한다. 본 시스템은 기초 조합 논리 회로(Combinational Logic Circuit)인 멀티플렉서(Mux), 디멀티플렉서(DMux)의 텐서곱(Tensor Product) 기반 일반화 표현식에서 출발하여, 16비트 산술 논리 장치(ALU)의 전처리, 연산, 후처리 상태 방정식을 체계적으로 정립한다. 또한, D 플립플롭(D Flip-Flop) 기반의 순차 논리 회로(Sequential Logic Circuit) 및 클럭 신호 동기화를 통해 레지스터 및 계층적 RAM 구조(RAM8부터 RAM16K까지)를 모듈화하여 합성한다. 최종적으로 메모리 맵 입출력(Memory-Mapped I/O) 구조와 A/C-명령어 디코더, 점프 조건 평가 로직을 포함하는 16비트 중앙 처리 장치(CPU)를 완성하고, 프로그램 카운터(PC) 및 ROM32K와의 결합을 통해 톱 레벨 단일 칩 컴퓨터 시스템을 구조적 HDL 코드로 구현 및 명세한다.

Index Terms—16-Bit Computer Architecture, Arithmetic Logic Unit (ALU), Memory-Mapped I/O, Hardware Description Language (HDL), Logic Synthesis, Sequential Logic, Cloud Computing Baseline.

I. 서론 (INTRODUCTION)

현대 클라우드 컴퓨팅 인프라스트럭처는 대규모 가상화 및 하드웨어 가속기 고도화를 요구함에 따라, 기본이 되는 프로세서 아키텍처의 형식적 동작 검증과 신뢰성 있는 회로 합성이 필수적이다. 본 연구에서는 클라우드 인프라 구축 프로젝트의 하위 논리 레이어 베이스라인을 제공하기 위해, 폰 노이만 아키텍처(Von Neumann Architecture) 기반의 16비트 단일 주기 컴퓨터 구조를 설계하고 불 대수(Boolean Algebra) 및 상태 방정식으로 명세한다.

본 논문의 구성은 다음과 같다. II절에서는 Mux, DMux, 가산기(Adder) 및 ALU를 포함하는 조합 논리 회로를 불 방정식과 수학적 텐서곱으로 정립한다. III절에서는 D 플립플롭 기반 기억 소자 및 RAM 계층 구조의 순차 논리 방정식을 다룬다. IV절에서는 메모리 맵 I/O, 명령어 디코딩 로직을 탑재한 CPU와 전체 시스템의 HDL 구현 명세를 제시하며, V절에서 결론을 맺는다.

II. 조합 논리 회로 및 ALU 설계 (COMBINATIONAL LOGIC & ALU ARCHITECTURE)

조합 논리 회로는 현재 입력값에만 기반하여 출력을 결정하는 논리 소자로, 데이터 전송 제어, 산술 연산, 논리 연산을 담당한다.

A. 기본 게이트 및 신호 선택 회로

1) *Multiplexer (Mux) & Demultiplexer (DMux)*: 멀티플렉서는 n 비트 선택 신호 S 를 통해 2^n 개의 입력 중 하나를 선택하여 출력하는 조합 논리 회로이다. 1비트 2-to-1 Mux의 입력 A, B 및 선택 신호 S 에 대한 불 방정식은 식 (1)과 같다.

$$\text{Mux}(A, B, S) = SB + \bar{S}A \quad (1)$$

여기서 $S = 0$ 이면 A 를, $S = 1$ 이면 B 를 출력한다.

디멀티플렉서는 하나의 입력 신호 I 와 n 비트 선택 신호 S 를 받아 2^n 개의 출력 라인 중 하나로 신호를 분배한다. 1비트 1-to-2 DMux의 출력쌍은 식 (2)로 정의된다.

$$\text{DMux}(I, S) = (\bar{S}I, SI) \quad (2)$$

$S = 0$ 일 때 첫 번째 출력 라인에 I 가 전달되고, $S = 1$ 일 때 두 번째 출력 라인에 I 가 전달된다.

2) *16-비트 게이트 및 대용량 멀티플렉서 트리*: 16비트 신호 벡터 $\mathbf{A}, \mathbf{B} \in \{0, 1\}^{16}$ 에 대해 NOT, AND, OR 게이트는 각 비트 위치 $i \in \{0, 1, \dots, 15\}$ 에 대하여 독립적으로 연산된다.

$$\text{NOT}_i(\mathbf{A}) = \bar{A}_i \quad (3)$$

$$\text{AND}_i(\mathbf{A}, \mathbf{B}) = A_i B_i \quad (4)$$

$$\text{OR}_i(\mathbf{A}, \mathbf{B}) = A_i + B_i \quad (5)$$

$$\text{Mux}_i^{16}(\mathbf{A}, \mathbf{B}, S) = \text{Mux}(A_i, B_i, S) \quad (6)$$

Or8Way 게이트는 8비트 입력 $\mathbf{A} \in \{0, 1\}^8$ 의 모든 비트에 대하여 비트 논리합(OR) 연산을 수행한다.

$$\text{Or8Way}(\mathbf{A}) = \sum_{i=0}^7 A_i = A_0 + A_1 + \dots + A_7 \quad (7)$$

(단, 여기서 $+$ 연산자는 불 대수의 논리합 OR을 의미한다.)

4-way 및 8-way 16비트 멀티플렉서는 2-to-1 Mux를 계층적으로 조합하여 합성된다. 선택 신호 $\mathbf{S} = (S_1, S_0)$ 에 대한 Mux4Way16과 선택 신호 $\mathbf{S} = (S_2, S_1, S_0)$ 에 대한 Mux8Way16의 재귀적 정의는 식 (8) 및 식 (9)와 같다.

$$\begin{aligned} \text{Mux4Way16}(A, B, C, D, \mathbf{S}) = & \\ & \text{Mux}^{16}(\text{Mux}^{16}(A, B, S_0), \\ & \text{Mux}^{16}(C, D, S_0), S_1) \end{aligned} \quad (8)$$

$$\begin{aligned} \text{Mux8Way16}(A, B, C, D, E, F, G, H, \mathbf{S}) = \\ \text{Mux}^{16} \left(\text{Mux4Way16}(A, B, C, D, \mathbf{S}[0..1]), \right. \\ \left. \text{Mux4Way16}(E, F, G, H, \mathbf{S}[0..1]), \mathbf{S}[2] \right) \end{aligned} \quad (9)$$

3) 크로네커 텐서곱(Kronecker Tensor Product) 기반 DMux 일반화 명세: 디멀티플렉서의 다중 출력 신호 분배는 스칼라 입력 I 와 선택 비트 벡터 선택 행렬 간의 텐서곱 $\otimes$ 으로 정밀하게 일반화할 수 있다. DMux4Way 및 DMux8Way의 대수적 표현은 다음과 같다.

$$\text{DMux4Way}(I, \mathbf{S}) = I \cdot \begin{pmatrix} \bar{S}_1 \\ S_1 \end{pmatrix} \otimes \begin{pmatrix} \bar{S}_0 \\ S_0 \end{pmatrix} \quad (10)$$

$$\text{DMux8Way}(I, \mathbf{S}) = I \cdot \begin{pmatrix} \bar{S}_2 \\ S_2 \end{pmatrix} \otimes \begin{pmatrix} \bar{S}_1 \\ S_1 \end{pmatrix} \otimes \begin{pmatrix} \bar{S}_0 \\ S_0 \end{pmatrix} \quad (11)$$

일반적으로 n 비트 주소 신호 $\mathbf{S} = (S_{n-1}, \dots, S_0)$ 를 갖는 2^n -way 디멀티플렉서의 출력 벡터는 식 (12)과 같이 compact 형식으로 정의된다.

$$\text{DMux}_{2^n}(I, \mathbf{S}) = I \cdot \bigotimes_{i=1}^n \begin{pmatrix} \bar{S}_{n-i} \\ S_{n-i} \end{pmatrix} \quad (12)$$

B. 산술 연산 장치 (Adder & Incrementer)

1) 반가산기(HalfAdder) 및 전가산기(FullAdder): 반가산기는 두 1비트 입력 A, B 의 캐리(Carry)와 합(Sum)을 연산한다.

$$\text{HalfAdder}(A, B) = (AB, A \oplus B) \quad (13)$$

전가산기는 하위 비트 캐리 C 를 포함한 3개 비트의 합산을 수행하며, 두 개의 반가산기와 하나의 OR 게이트로 구성된다.

$$\begin{aligned} \text{FullAdder}(A, B, C) = \\ \left(\text{HA}_0(\text{HA}_0(A, B), C), \right. \\ \left. \text{HA}_1(A, B) + \text{HA}_1(\text{HA}_0(A, B), C) \right) \end{aligned} \quad (14)$$

단, HA_0 는 HalfAdder의 Sum 출력($\oplus$), HA_1 은 Carry 출력($\cdot$)을 의미한다.

2) 16-비트 가산기 및 증분기 (Add16 & Inc16): 16비트 Ripple-Carry Adder는 두 16비트 입력 $\mathbf{A}, \mathbf{B}$ 에 대하여 각 비트별 Carry Propagation을 거쳐 출력 $\mathbf{S}$ 와 캐리 출력 C 를 산출한다.

$$S_0 = A_0 \oplus B_0 \quad (15)$$

$$C_0 = A_0 B_0 \quad (16)$$

$$S_{i+1} = (A_{i+1} \oplus B_{i+1}) \oplus C_i \quad (0 \leq i \leq 14) \quad (17)$$

$$C_{i+1} = A_{i+1} B_{i+1} + C_i (A_{i+1} \oplus B_{i+1}) \quad (0 \leq i \leq 14) \quad (18)$$

$$C = C_{15} \quad (19)$$

16비트 증분기(Inc16)는 입력 $\mathbf{I}$ 에 1을 더하는 논리 회로로서 식 (21)으로 명세된다.

$$S_0 = \bar{I}_0 \quad (20)$$

$$S_{n+1} = I_{n+1} \oplus \prod_{i=0}^n I_i \quad (0 \leq n \leq 14) \quad (21)$$

$$C = \prod_{i=0}^{15} I_i \quad (22)$$

C. ALU (Arithmetic Logic Unit) 구조 및 논리 명세

본 설계의 16비트 ALU는 입력 신호 $\mathbf{x}, \mathbf{y} \in \{0, 1\}^{16}$ 과 6개의 제어 비트 ($z_x, n_x, z_y, n_y, f, n_o$)를 받아 16비트 연산 결과 $\mathbf{out}$ 과 1비트 상태 플래그 zr, ng 를 출력한다.

1) 전처리 단계 (Preprocessing): 입력 $\mathbf{x}, \mathbf{y}$ 는 zeroing 제어 비트 z_x, z_y 와 negation 제어 비트 n_x, n_y 에 의해 변형된 중간 신호 $\mathbf{x}', \mathbf{y}'$ 로 변환된다 ($0 \leq i \leq 15$).

$$x'_i = n_x(z_x + \bar{x}_i) + x_i \bar{n}_x \bar{z}_x \quad (23)$$

$$y'_i = n_y(z_y + \bar{y}_i) + y_i \bar{n}_y \bar{z}_y \quad (24)$$

2) 연산 및 산술 전파 (Core Arithmetic/Logic Computation): 가산기 전파 연산 신호 s_i 와 캐리 신호 c_i 는 식 (26)와 같다.

$$s_0 = x'_0 \oplus y'_0, \quad c_0 = x'_0 y'_0 \quad (25)$$

$$c_{i+1} = x'_{i+1} y'_{i+1} + c_i (x'_{i+1} \oplus y'_{i+1}) \quad (0 \leq i \leq 13) \quad (26)$$

$$s_{i+1} = (x'_{i+1} \oplus y'_{i+1}) \oplus c_i \quad (0 \leq i \leq 14) \quad (27)$$

제어 비트 f 에 따라 비트 논리곱($f=0$)과 산술 가산($f=1$) 중 하나를 선택하여 중간 연산 결과 z_i 를 구한다.

$$z_i = \bar{f} x'_i y'_i + f s_i \quad (0 \leq i \leq 15) \quad (28)$$

3) 후처리 및 상태 플래그 도출 (Output Negation & Flag Generation): 최종 출력 out_i 는 출력 반전 비트 n_o 에 의해 결정된다. Zero 플래그 zr 은 모든 출력 비트가 0일 때 1이 되며, Negative 플래그 ng 는 최상위 비트(MSB, Sign Bit) out_{15} 를 복사한다.

$$out_i = n_o \oplus z_i \quad (0 \leq i \leq 15) \quad (29)$$

$$zr = \prod_{i=0}^{15} \overline{out_i} \quad (30)$$

$$ng = out_{15} \quad (31)$$

주요 6비트 제어 신호에 따른 대표적 연산 매핑은 표 I와 같다.

TABLE I
ALU 제어 비트에 따른 연산 기능 매핑

z_x	n_x	z_y	n_y	f	n_o	출력 연산 (out)
1	0	1	0	1	0	0
1	1	1	1	1	1	1
1	1	1	0	1	0	-1
0	0	1	1	0	0	x
1	1	0	0	0	0	y
0	0	1	1	0	1	$!x$
1	1	0	0	0	1	$!y$
0	0	1	1	1	1	$-x$
1	1	0	0	1	1	$-y$
0	1	1	1	1	1	$x + 1$
1	1	0	1	1	1	$y + 1$
0	0	1	1	1	0	$x - 1$
1	1	0	0	1	0	$y - 1$
0	0	0	0	1	0	$x + y$
0	1	0	0	1	1	$x - y$
0	0	0	1	1	1	$y - x$
0	0	0	0	0	0	$x \& y$
0	1	0	1	0	1	$x y$

III. 순차 논리 회로 및 메모리 계층 구조 (SEQUENTIAL LOGIC & MEMORY HIERARCHY)

순차 논리 회로는 클럭(Clock) 신호에 맞춰 동기화되며, 내부 상태(State)를 보존함으로써 기억 장치를 형성한다. 본 시스템의 모든 기억 소자는 Positive Edge-Triggered 클럭 방식을 따른다.

A. 1-비트 메모리 셀(Bit) 및 16-비트 레지스터(Register)

1) *Bit 셀의 상태 방정식*: 1비트 기억 소자 Bit는 D 플립플롭을 기반으로 제작되며, *load* 제어 신호에 따라 이전 상태 $Q(t)$ 를 유지하거나 입력 *in*을 새로 저장한다. 클럭 활성화 상태($CP = 1$)에서의 차기 상태 방정식은 식 (32)과 같다.

$$Q(t+1) = \overline{load} \cdot Q(t) + load \cdot in \quad (32)$$

클럭 신호 CP (Clock Pulse)를 명시적으로 합성한 완전한 물리 상태 방정식은 다음과 같이 기술된다.

$$Q(t+1) = CP \cdot (\overline{load} \cdot Q(t) + load \cdot in) + \overline{CP} \cdot Q(t) \quad (33)$$

2) *16-비트 레지스터 (Register)*: 16비트 레지스터는 16개의 Bit 셀을 병렬 연결하여 구현되며, 각 비트 $i \in \{0, \dots, 15\}$ 의 차기 상태 방정식은 다음과 같다.

$$Q_i(t+1) = \overline{load} \cdot Q_i(t) + load \cdot in_i \quad (0 \leq i \leq 15) \quad (34)$$

B. 계층적 RAM 구조 (Hierarchical RAM Architecture)

메모리는 주소(Address) 신호와 디멀티플렉서/멀티플렉서 트리를 이용해 상위 계층으로 확장된다.

- 1) **RAM8 (8-Word Memory)**: 3비트 주소를 통해 8개의 Register 중 하나를 선택하여 읽기/쓰기를 수행한다.
- 2) **RAM64 (64-Word Memory)**: 6비트 주소 중 상위 3비트($address[3..5]$)로 8개의 RAM8 블록 중 하나를

지정하고, 하위 3비트($address[0..2]$)를 각 RAM8의 내부 주소로 전달한다.

- 3) **RAM512, RAM4K, RAM16K**: 동일한 재귀 계층 구조를 확장하여 512, 4K, 16K 단어를 수용하는 RAM 단위를 합성한다.

Listing 1과 Listing 2는 각각 RAM8과 RAM64의 칩 구조를 보여주는 HDL 코드 명세이다.

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
        // Step 1: Distribute load signal to 8 registers based
        // on address
        DMux8Way(in=load, sel=address, a=load0, b=load1, c=
            load2, d=load3, e=load4, f=load5, g=load6, h=load7
        );

        // Step 2: Input data into 8 registers (shared data bus
        // 'in')
        Register(in=in, load=load0, out=reg0);
        Register(in=in, load=load1, out=reg1);
        Register(in=in, load=load2, out=reg2);
        Register(in=in, load=load3, out=reg3);
        Register(in=in, load=load4, out=reg4);
        Register(in=in, load=load5, out=reg5);
        Register(in=in, load=load6, out=reg6);
        Register(in=in, load=load7, out=reg7);

        // Step 3: Output the selected register specified by
        // address
        Mux8Way16(a=reg0, b=reg1, c=reg2, d=reg3, e=reg4, f=
            reg5, g=reg6, h=reg7, sel=address, out=out);
}
```

Listing 1. RAM8 모듈의 Hardware Description Language 명세

```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
        DMux8Way(in=load, sel=address[3..5], a=ram0, b=ram1, c=
            ram2, d=ram3, e=ram4, f=ram5, g=ram6, h=ram7);

        RAM8(in=in, load=ram0, address=address[0..2], out=out0)
        ;
        RAM8(in=in, load=ram1, address=address[0..2], out=out1)
        ;
        RAM8(in=in, load=ram2, address=address[0..2], out=out2)
        ;
        RAM8(in=in, load=ram3, address=address[0..2], out=out3)
        ;
        RAM8(in=in, load=ram4, address=address[0..2], out=out4)
        ;
        RAM8(in=in, load=ram5, address=address[0..2], out=out5)
        ;
        RAM8(in=in, load=ram6, address=address[0..2], out=out6)
        ;
        RAM8(in=in, load=ram7, address=address[0..2], out=out7)
        ;

        Mux8Way16(a=out0, b=out1, c=out2, d=out3, e=out4, f=
            out5, g=out6, h=out7, sel=address[3..5], out=out);
}
```

Listing 2. RAM64 계층 합성 모듈 명세

대용량 16KB RAM 모듈인 RAM16K는 4개의 RAM4K 칩과 2비트 상위 주소($address[12..13]$)를 통한 DMux4Way/Mux4Way16 제어로 구현된다 (Listing 3).

```
CHIP RAM16K {
    IN in[16], load, address[14];
    OUT out[16];

    PARTS:
```

```

DMux4Way (in=load, sel=address[12..13], a=ram0, b=ram1,
c=ram2, d=ram3);

RAM4K (in=in, load=ram0, address=address[0..11], out=
out0);
RAM4K (in=in, load=ram1, address=address[0..11], out=
out1);
RAM4K (in=in, load=ram2, address=address[0..11], out=
out2);
RAM4K (in=in, load=ram3, address=address[0..11], out=
out3);

Mux4Way16 (a=out0, b=out1, c=out2, d=out3, sel=address
[12..13], out=out);
}

```

Listing 3. RAM16K 대용량 메모리 모듈 명세

IV. 프로세서 및 통합 컴퓨터 시스템 (CPU & SYSTEM INTEGRATION)

A. 메모리 맵 입출력 구조 (Memory-Mapped I/O)

전체 32K 단어(15비트 주소 공간)의 주소 영역 분할 명세는 표 II과 같다.

TABLE II
SYSTEM MEMORY-MAPPED I/O ADDRESS SPACE

주소 범위	비트 13..14	매핑 장치	용도 및 특징
0x0000 ~ 0x3FFF	00, 01	RAM16K	데이터/스택 메모리 (16K Words)
0x4000 ~ 0x5FFF	10	Screen	8K 비디오 비트맵 (Read/Write)
0x6000	11	Keyboard	실시간 키 스캔코드 (Read-only)

이를 관장하는 Memory 칩의 HDL 구체화 코드는 Listing 4와 같다.

```

CHIP Memory {
    IN in[16], load, address[15];
    OUT out[16];

    PARTS:
        // Step 1: Route load control signal based on top 2
        address bits (13..14)
        DMux4Way (in=load, sel=address[13..14], a=loadRam1, b=
        loadRam2, c=loadScreen, d=loadKbd);

        // RAM16K covers both 00 and 01 address ranges
        Or (a=loadRam1, b=loadRam2, out=loadRam);

        // Step 2: Connect main RAM, Screen, and Keyboard
        devices
        RAM16K (in=in, load=loadRam, address=address[0..13], out
        =ramOut);
        Screen (in=in, load=loadScreen, address=address[0..12],
        out=screenOut);
        Keyboard (out=kbdOut);

        // Step 3: Select final output device based on top 2
        address bits
        Mux4Way16 (a=ramOut, b=ramOut, c=screenOut, d=kbdOut,
        sel=address[13..14], out=out);
}

```

Listing 4. Memory-Mapped I/O 통합 Memory 모듈 HDL

B. 16-비트 중앙 처리 장치 (CPU Design & Control Unit)

CPU는 16비트 명령어(instruction[16])를 해석하여 산술 연산, 레지스터 쓰기, 메모리 접근 및 조건부 점프 제어를 수행한다.

1) 명령어 디코딩 (Instruction Decoding): 최상위 비트 instruction[15]가 0이면 A-명령어(주소 로드), 1이면 C-명령어(연산 및 제어)로 디코딩된다.

$$isA = \overline{instruction[15]} \quad (35)$$

$$isC = instruction[15] \quad (36)$$

2) 레지스터 제어 logic (loadA, loadD, writeM):

- **A-Register:** A-명령어이거나, C-명령어이면서 Destination 비트 instruction[5]가 1일 때 로드된다.

$$destA = isC \cdot instruction[5] \quad (37)$$

$$loadA = isA + destA \quad (38)$$

- **D-Register:** C-명령어이면서 instruction[4]가 1일 때 로드된다.

$$loadD = isC \cdot instruction[4] \quad (39)$$

- **Memory Write Signal (writeM):** C-명령어이면서 instruction[3]이 1일 때 쓰기 신호가 활성화된다.

$$writeM = isC \cdot instruction[3] \quad (40)$$

3) ALU 입력 선택 및 Jump 제어 로직: ALU의 y 입력은 instruction[12] 비트('a' 비트)에 의해 A-레지스터값과 메모리 입력값 inM 중 하나가 선택된다.

$$AM = Mux16(outA, inM, instruction[12]) \quad (41)$$

분기 조건 판단 로직은 ALU 플래그 zr, ng 및 명령어 분기 비트 instruction[0..2]를 조합하여 도출된다.

$$pos = \overline{zr} + ng \quad (\text{양수 조건: } > 0) \quad (42)$$

$$jlt = instruction[2] \cdot ng \quad (\text{Jump if } < 0) \quad (43)$$

$$jeq = instruction[1] \cdot zr \quad (\text{Jump if } = 0) \quad (44)$$

$$jgt = instruction[0] \cdot pos \quad (\text{Jump if } > 0) \quad (45)$$

$$jumpCond = jlt + jeq + jgt \quad (46)$$

$$loadPC = isC \cdot jumpCond \quad (47)$$

프로그램 카운터(PC)는 loadPC = 1일 때 A-레지스터값으로 분기하며, 그렇지 않으면 자동으로 1씩 증분(inc = true)한다. 전체 CPU 칩의 상세 HDL 구조는 Listing 5에 명세되어 있다.

```

CHIP CPU {
    IN inM[16], instruction[16], reset;
    OUT outM[16], writeM, addressM[15], pc[15];

    PARTS:
        // Step 1: Decode A-instruction vs C-instruction
        Not (in=instruction[15], out=isA);
        Not (in=isA, out=isC);

        // Step 2: Select input for A-register (instruction vs
        ALU output)
        Mux16 (a=instruction, b=ALUout, sel=isC, out=inA);

        // Step 3: A-register load condition and storage
        And (a=isC, b=instruction[5], out=destA);
        Or (a=isA, b=destA, out=loadA);
        ARegister (in=inA, load=loadA, out=outA, out[0..14]=
        addressM);

        // Step 4: Select ALU y-input (A-register vs Memory
        input)
        Mux16 (a=outA, b=inM, sel=instruction[12], out=AM);
}

```

```

// Step 5: D-register load condition and storage
And(a=isC, b=instruction[4], out=loadD);
DRegister(in=ALUout, load=loadD, out=outD);

// Step 6: Execute ALU arithmetic/logical operations
ALU(x=outD, y=AM,
    zx=instruction[11], nx=instruction[10],
    zy=instruction[9], ny=instruction[8],
    f=instruction[7], no=instruction[6],
    out=ALUout, out=outM, zr=zr, ng=ng);

// Step 7: Derive memory write signal
And(a=isC, b=instruction[3], out=writeM);

// Step 8: Evaluate jump conditions
Or(a=zr, b=ng, out=zrOrNg);
Not(in=zrOrNg, out=pos);

And(a=instruction[2], b=ng, out=jlt);
And(a=instruction[1], b=zr, out=jeq);
And(a=instruction[0], b=pos, out=jgt);

Or(a=jlt, b=jeq, out=jle);
Or(a=jle, b=jgt, out=jumpCond);
And(a=isC, b=jumpCond, out=loadPC);

// Step 9: Program Counter (PC) update
PC(in=outA, load=loadPC, inc=true, reset=reset, out
    [0..14]=pc);
}

```

Listing 5. 16비트 CPU 중앙 처리 장치 HDL 명세

C. 톱 레벨 통합 컴퓨터 (Top-Level Computer Integration)

최종 컴퓨터 시스템(Computer 01)은 ROM32K(명령어 메모리), CPU, 그리고 Memory(데이터 및 I/O 메모리) 칩을 단일 버스로 상호연결하여 완결된 단일 칩 컴퓨터를 형성한다 (Listing 6).

```

CHIP Computer {
    IN reset;

    PARTS:
    // 1. ROM32K: Outputs instruction based on PC value
    ROM32K(address=pc, out=instruction);

    // 2. CPU: Executes operation and outputs control
    // signals & next PC
    CPU(inM=memOut, instruction=instruction, reset=reset,
        outM=outM, writeM=writeM, addressM=addressM, pc=pc)
        ;

    // 3. Memory: Performs read/write based on CPU control
    // signals
    Memory(in=outM, load=writeM, address=addressM, out=
        memOut);
}

```

Listing 6. 톱 레벨 단일 칩 컴퓨터 모듈 HDL 명세

V. 결론 (CONCLUSION)

본 논문에서는 클라우드 컴퓨팅 환경의 하드웨어 베이스 라인 확립을 위하여 16비트 단일 주기 컴퓨터 구조를 하위 기본 게이트 수준부터 톱 레벨 통합 컴퓨터 시스템까지 수학적 불 방정식과 계층적 HDL 명세로 완전하게 정립하였다. Mux/DMux 트리의 텐서곱 기반 일반화 표현식을 도출하였으며, 6비트 제어를 통한 ALU 정밀 연산 및 플래그 생성 로직을 규명하였다. 또한 15비트 메모리 맵 입출력 체계와 A/C-명령어 디코딩 및 조건부 점프 회로가 탑재된 16비트 CPU의 올바른 동작 구성을 입증하였다.

본 설계 명세는 향후 클라우드 가상화용 가스텀 처리 장치, FPGA 기반 논리 합성 및 도메인 특화 가속기(DSA) 구축

프로젝트의 신뢰성 높은 하드웨어 명세 기준으로 활용될 수 있다.

REFERENCES

- [1] N. Nisan and S. Schocken, *The Elements of Computing Systems: Building a Modern Computer from First Principles*, 2nd ed. Cambridge, MA, USA: MIT Press, 2021.
- [2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 6th ed. Morgan Kaufmann, 2020.
- [3] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2017.